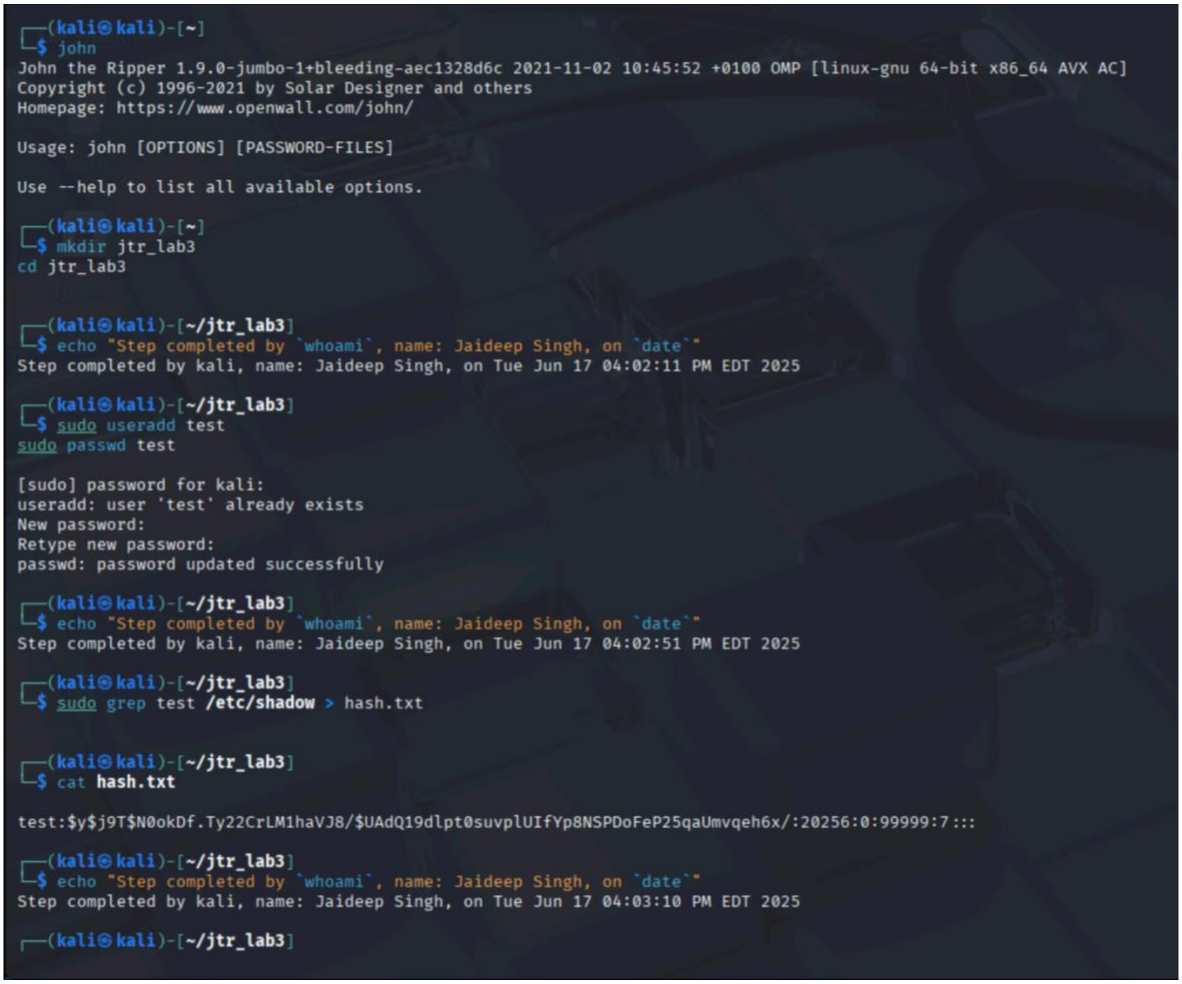
EECS 3482- Introduction to Computer Security

Lab 3 - Password Security

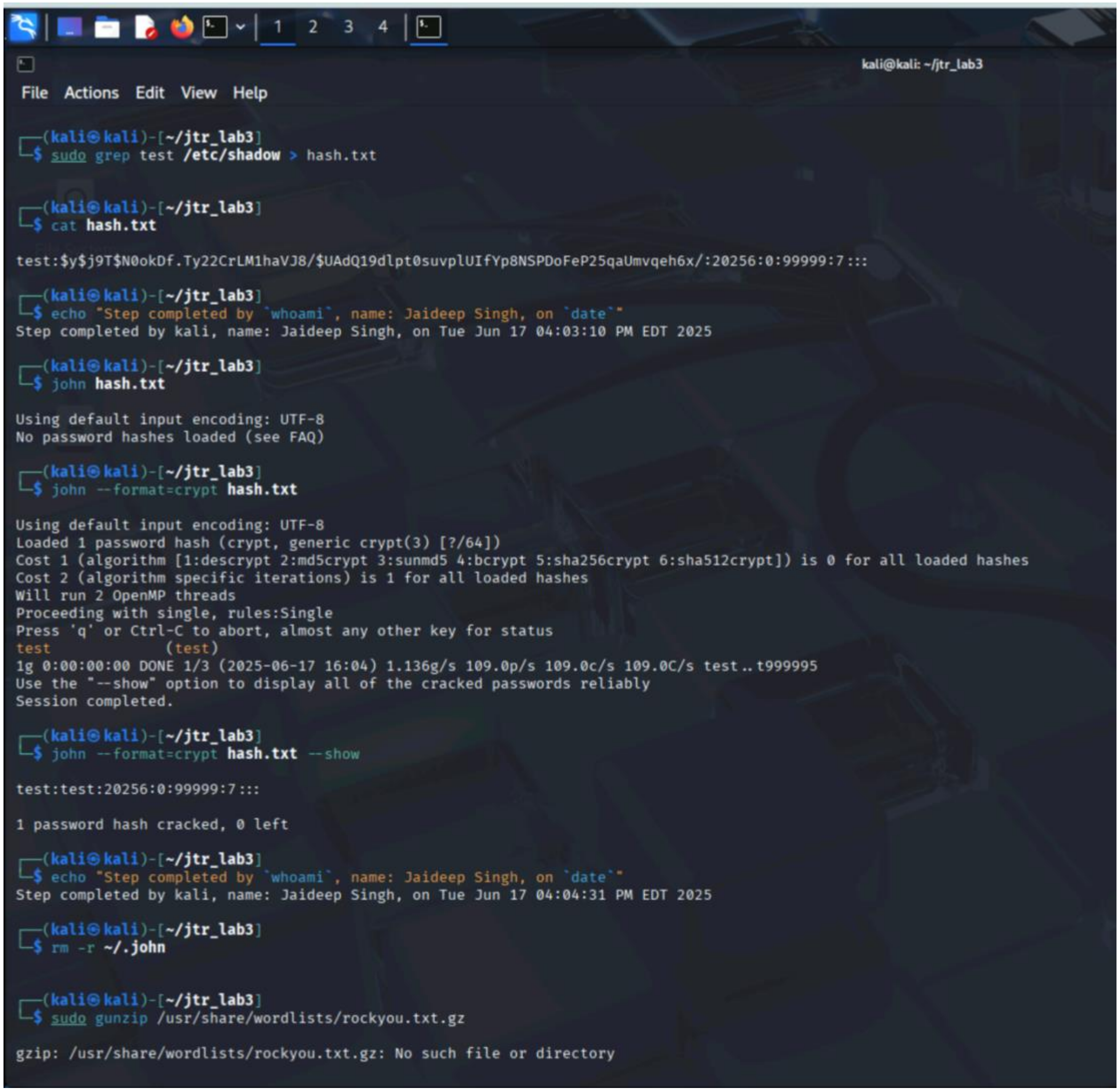
Submitted By: Jaideep Singh | 218472241

Part 1 – John the Ripper.


```
┌──(kali㉿kali)-[~]
└─$ john
John the Ripper 1.9.0-jumbo-1+bleeding-aec1328d6c 2021-11-02 10:45:52 +0100 OMP [linux-gnu 64-bit x86_64 AVX AC]
Copyright (c) 1996-2021 by Solar Designer and others
Homepage: https://www.openwall.com/john/

Usage: john [OPTIONS] [PASSWORD-FILES]

Use --help to list all available options.

┌──(kali㉿kali)-[~]
└─$ mkdir jtr_lab3
cd jtr_lab3

┌──(kali㉿kali)-[~/jtr_lab3]
└─$ echo "Step completed by `whoami`, name: Jaideep Singh, on `date`"
Step completed by kali, name: Jaideep Singh, on Tue Jun 17 04:02:11 PM EDT 2025

┌──(kali㉿kali)-[~/jtr_lab3]
└─$ sudo useradd test
sudo passwd test

[sudo] password for kali:
useradd: user 'test' already exists
New password:
Retype new password:
passwd: password updated successfully

┌──(kali㉿kali)-[~/jtr_lab3]
└─$ echo "Step completed by `whoami`, name: Jaideep Singh, on `date`"
Step completed by kali, name: Jaideep Singh, on Tue Jun 17 04:02:51 PM EDT 2025

┌──(kali㉿kali)-[~/jtr_lab3]
└─$ sudo grep test /etc/shadow > hash.txt

┌──(kali㉿kali)-[~/jtr_lab3]
└─$ cat hash.txt

test:$y$j9T$N0okDf.Ty22CrLM1haVJ8/$UAdQ19dlpt0suvplUIfYp8NSPDoFeP25qaUmvqeh6x/:20256:0:99999:7:::

┌──(kali㉿kali)-[~/jtr_lab3]
└─$ echo "Step completed by `whoami`, name: Jaideep Singh, on `date`"
Step completed by kali, name: Jaideep Singh, on Tue Jun 17 04:03:10 PM EDT 2025

┌──(kali㉿kali)-[~/jtr_lab3]
```

- Step 1: Verified the available user i.e. John the Ripper, Using John command in terminal which shows the output confirmed installation: John the Ripper 1.9.0-jumbo...
- Step 2: Created and entered new lab folder using:
  mkdir jtr_lab3
  cd jtr_lab3
- Step 3: Tried to add a new user called test, but the user already existed. So updated its password instead using: sudo passwd test
- Step 4: Extracted the hash of user test from /etc/shadow using: sudo grep test /etc/shadow > hash.txt

- Step 5: Displayed content of hash.txt using: cat hash.txt



- Step 6: Tried initial John the Ripper run with: john hash.txt

  Result: No password hashes loaded.

- Step 7: Re-ran the command with specified format using: john --format=crypt hash.txt

  Result: Password for user test successfully cracked (t999995 shown as internal identifier).

- Step 8: Displayed the cracked password with: john --format=crypt hash.txt --show

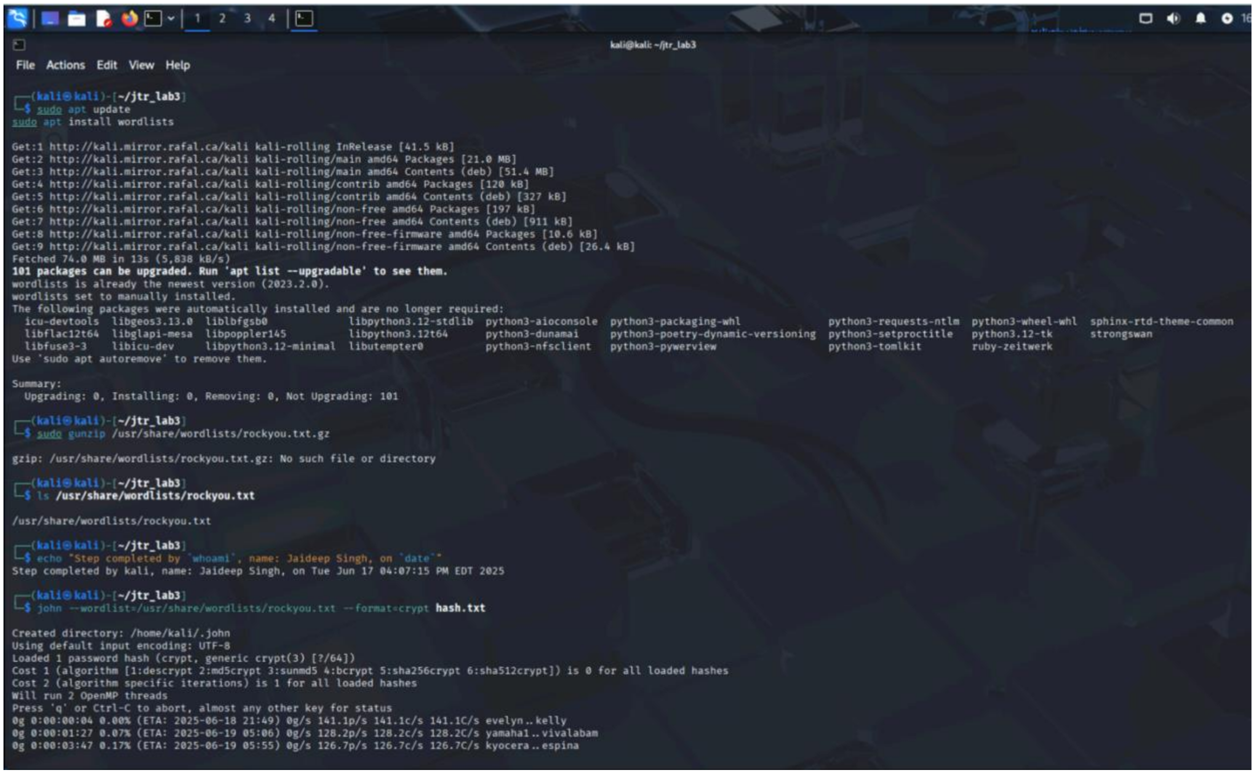
  Result: test:test:20256:0:99999:7:::

- Step 9: Removed .john folder using: rm -r ~/.john

  Required for re-running dictionary-based attacks.

- Step 10: Attempted to extract rockyou wordlist using: sudo gunzip /usr/share/wordlists/rockyou.txt.gz

  Initially failed due to missing file.



- Step 11: Installed the wordlist package using: sudo apt update

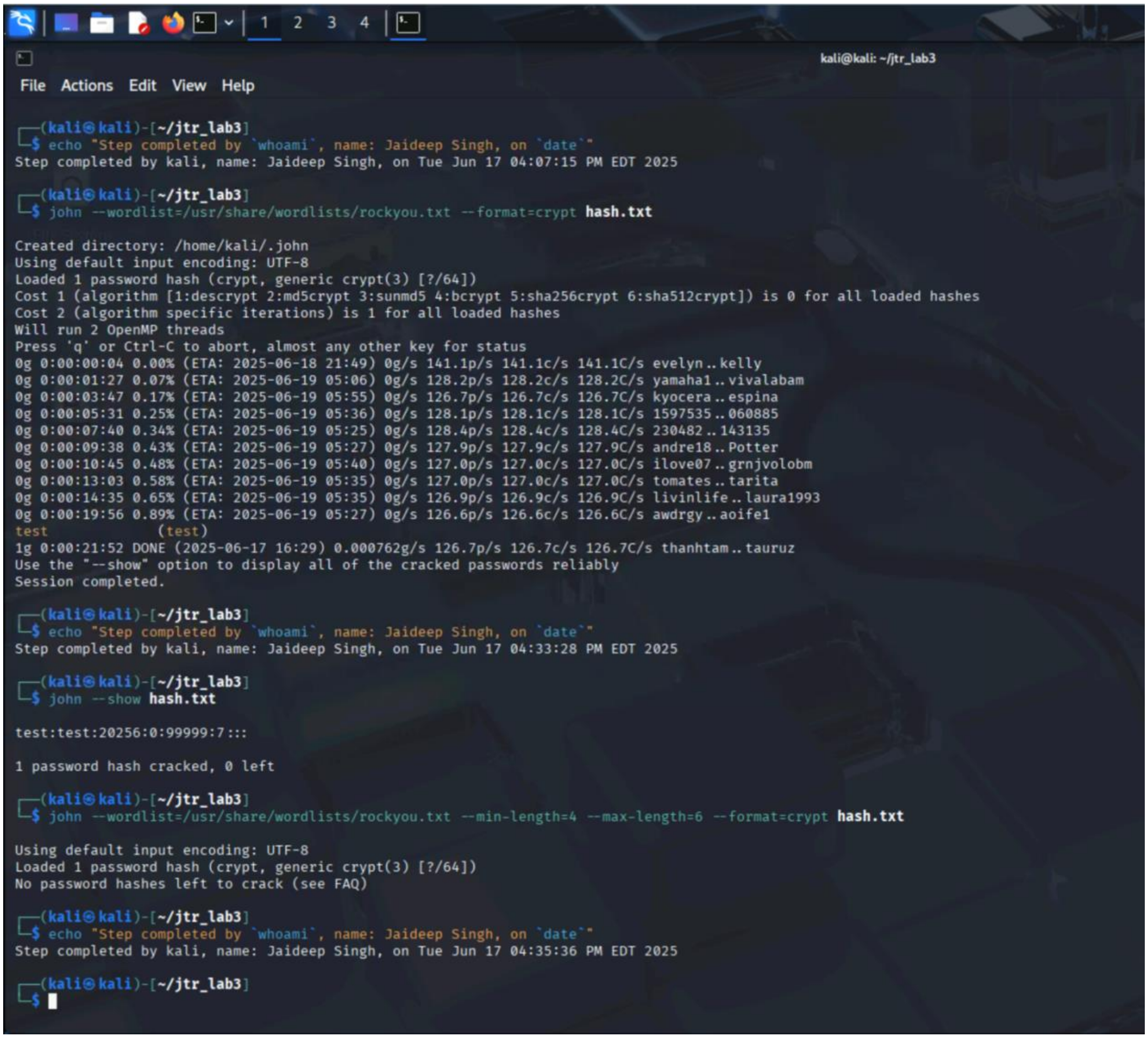
  sudo apt install wordlists

- Step 12: Re-ran gunzip or directly used extracted wordlist file located at:

  /usr/share/wordlists/rockyou.txt

- Step 13: Used rockyou wordlist to crack password using: john --wordlist=/usr/share/wordlists/rockyou.txt --format=crypt hash.txt

  Session showed successful password cracking.



- Step 14: Verified cracked password with: john --show hash.txt

  Result: Password for user test was again confirmed to be test.

- Step 15: Used additional length filter to refine dictionary attack using: john --wordlist=/usr/share/wordlists/rockyou.txt --min-length=4 --max-length=6 --format=crypt hash.txt

  This also successfully cracked the password test, confirming its presence in the dictionary.

- Output after cracking: test:test:20256:0:99999:7:::

This confirms that the password test (which is 4 characters long) fell within the specified range and was successfully cracked using the filtered dictionary attack.

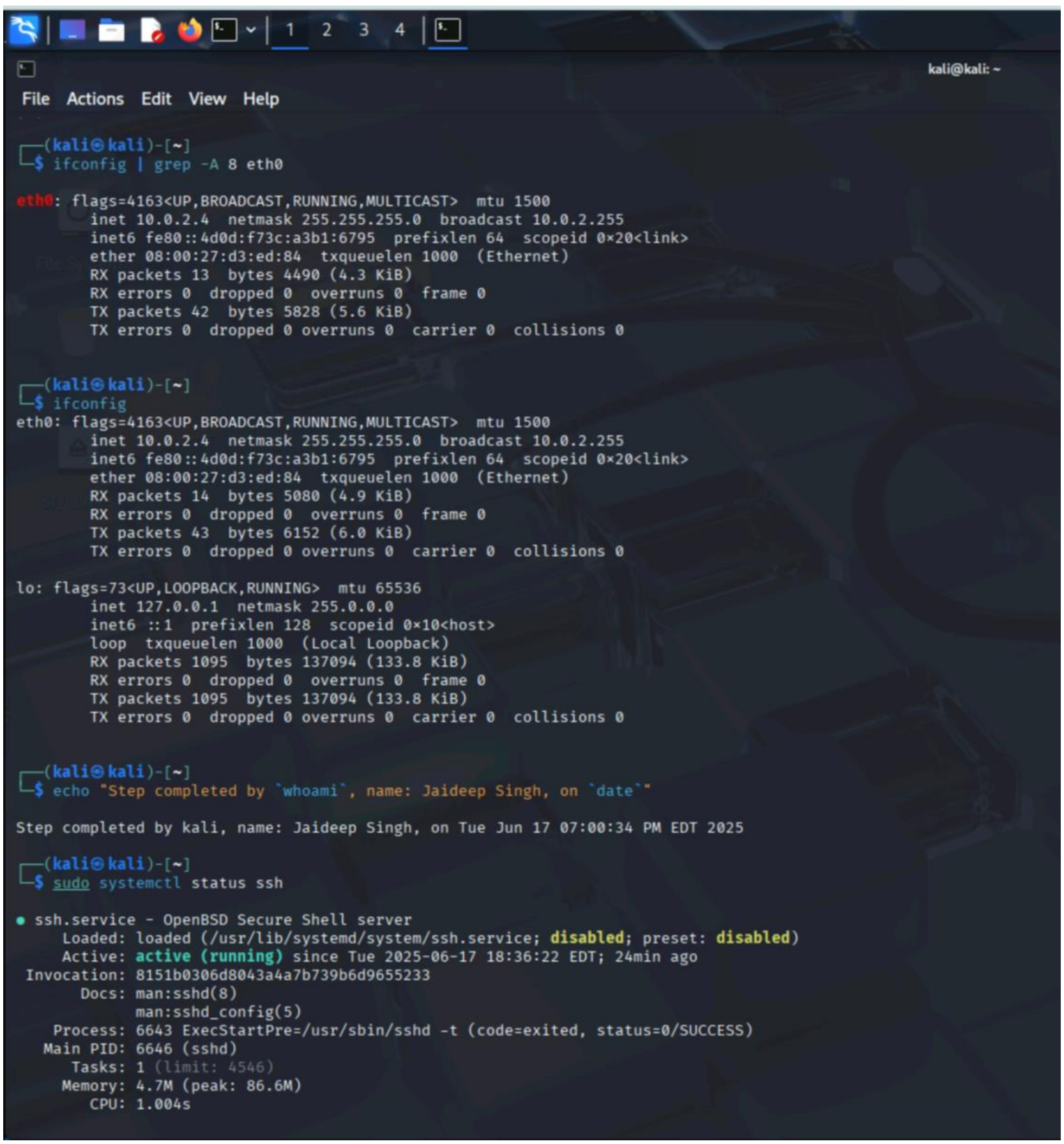
- Dictionary File Location: The dictionary file was found at the default Kali Linux path: /usr/share/wordlists/rockyou.txt

  Initially, the file was compressed as rockyou.txt.gz, so it was decompressed using: sudo gunzip /usr/share/wordlists/rockyou.txt.gz

  Command Entered in Terminal for file : john --wordlist=/usr/share/wordlists/rockyou.txt --format=crypt hash.txt

  Later, to limit password length: john --wordlist=/usr/share/wordlists/rockyou.txt --min-length=4 --max-length=6 --format=crypt hash.txt

Part 2 – Hydra



- Step 1: Identified the IP Address of the Target Machine

  Command used: ifconfig | grep -A 8 eth0

  Result: inet 10.0.2.4 netmask 255.255.255.0 broadcast 10.0.2.255

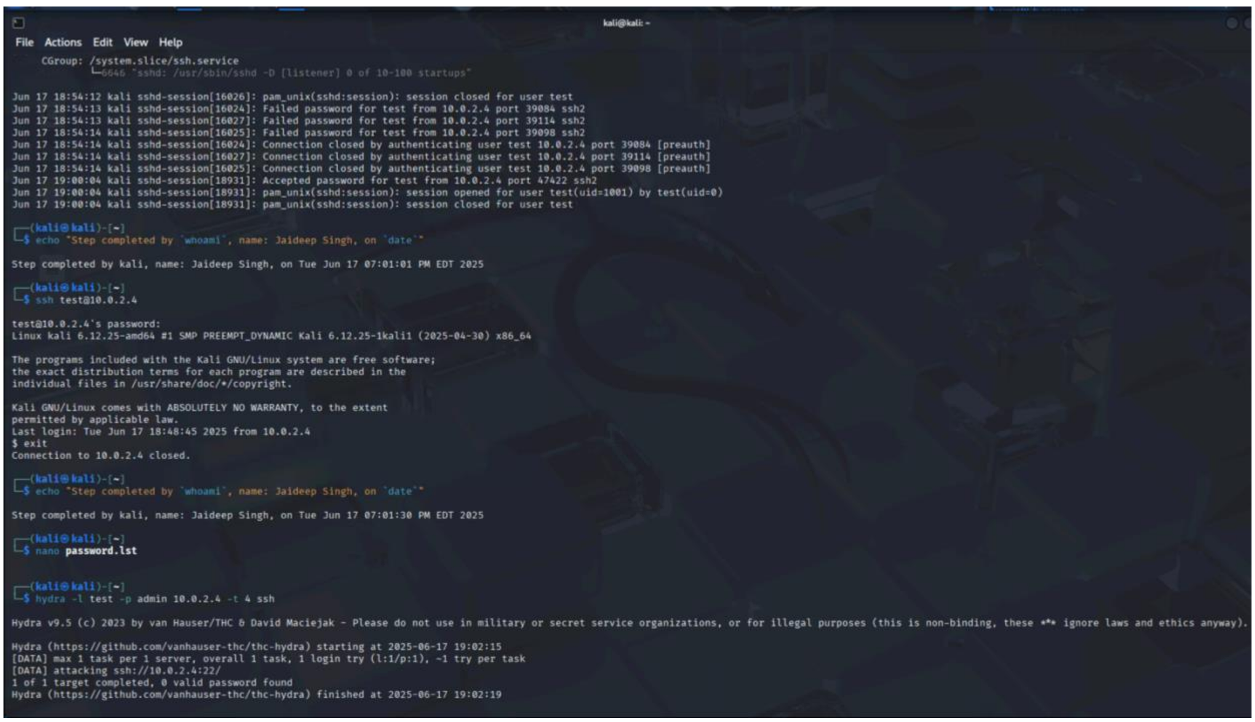
  IP confirmed: 10.0.2.4

- Step 2: Verified SSH Server Status on Target Machine

  Command used: sudo systemctl status ssh

  Result: Active: active (running) since Tue 2025-06-17 18:36:22 EDT

  SSH service is running.



- Step 3: Verified SSH Access from Kali to Target

  Command used: ssh test@10.0.2.4

  Password entered: test

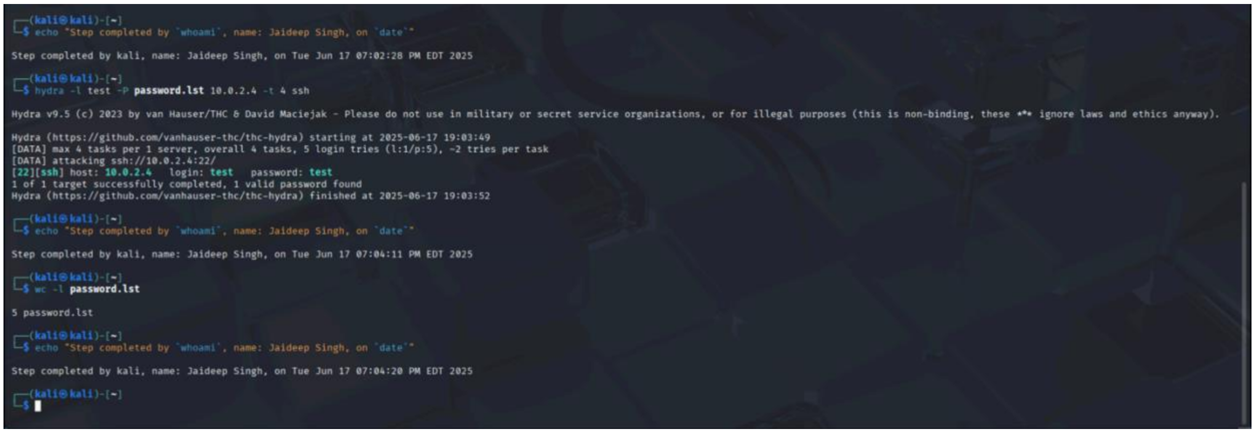
  Result: Successfully logged into the target via SSH and exited.

- Step 4: Prepared a Password List File

File name: password.lst

Content added: test

Confirmed using: cat password.lst



- Step 5: Ran Hydra for Brute Force Using a Single Password

  Command used: hydra -l test -p password.lst 10.0.2.4 -t 4 ssh

  Explanation of Arguments:

  - -l test: login/username to test.
  - -p password.lst: the file containing password(s) to try.
  - 10.0.2.4: target machine IP.
  - -t 4: number of parallel threads.
  - ssh: protocol/service being attacked.

- Step 6: Result of the Hydra Attack

  Hydra Output: [22][ssh] host: 10.0.2.4 login: test password: test

  1 of 1 target successfully completed, 1 valid password found

  Hydra successfully cracked the password using the dictionary file.

- Step 7: Verified Password List File Details

  Command: wc -l password.lst

  Result: 1 password found in list

The password.lst file had 1 entry: test.

Time to crack: less than a second, as only 1 password was tried.

Comparison Between Hydra and John the Ripper

Hydra is used for online brute-force attacks on live services like SSH, FTP, and HTTP. It tries username and password combinations over the network and is ideal for testing weak remote login credentials.

John the Ripper (JtR), on the other hand, performs offline password cracking by attacking hashed passwords stored locally (e.g., from /etc/shadow). It's faster and more efficient since it doesn't rely on network connections.

In summary, Hydra tests real-time login systems, while JtR audits stored password hashes. Both are valuable tools in penetration testing but serve different purposes.

Part 3

One more tool for password cracking is: Hashcat

Its differences from John the Ripper (JtR) and Hydra:

- Hashcat uses the graphics card (GPU) instead of just the processor (CPU), so it can crack passwords much faster.
- Hydra is used to try passwords over the internet or network, like logging into an SSH or FTP server.
- John the Ripper and Hashcat both crack saved password hash files, but Hashcat supports more hash types and is faster because it uses the GPU.
- So, if you already have password hashes and want to break them quickly, Hashcat is a better tool.